

Effective Agentic Coding for Scientific Computing

Patterns, pitfalls, and a live case study

James Kermode · School of Engineering, University of Warwick

15 June 2026 · Max Planck Institute for Polymer Research

Personal experience as a Warwick academic and HetSys CDT director — not the institutional view of the University of Warwick. Warwick's AI strategy is being developed separately.

A spectrum of AI coding tools

Vibe coding

increasing context · increasing autonomy · increasing oversight responsibility

Agentic engineering

ChatGPT in browser

- Copy/paste both ways
- No project context
- You run the code
- **ChatGPT, Claude.ai**

Autocomplete in editor

- Inline tab-completion
- Current-file context
- You drive every keystroke
- **Copilot, Cursor tab**

Full AI editor

- Chat edits files in repo
- Codebase-aware context
- You approve diffs in IDE
- **Cursor, Windsurf**

Agent

- Runs commands, iterates
- Full project context (CLAUDE.md)
- Review at task boundaries
- **Claude Code, Codex, Gemini CLI**

*Same model can sit behind any of these — the difference is the **harness**: how much context it sees, whether it can act, and where the human reviews. Willison: “Agentic Engineering represents the other end of the scale — professional engineers using coding agents to amplify their existing expertise.” Case studies that follow use Claude Code; the patterns themselves are agnostic to which agentic harness you prefer.*

How an LLM chatbot works

Pseudocode Sketch

```
messages = [system_prompt, task]

while not done:
    reply = model.generate(messages)

    messages.append(reply.text)
    display(reply.text)
    done = await_user()
```

Key properties

- ▶ **System prompt** – persistent instructions set by the application (role, tone, safety, format)
- ▶ **messages** – full conversation history, replayed on every call; the model has no memory between sessions
- ▶ **Context window** – finite tokens per turn (200k-1M tokens, ~150k-750k words)
- ▶ **Sampling** – `generate()` picks tokens probabilistically; same input can give different replies (temperature, top-p)
- ▶ **Knowledge cutoff** – no access to events after training ends
- ▶ **No actions** – model cannot run code, read files, or browse the web – only text in, text out

ChatGPT, Claude, Gemini etc. all basically work like this.

Agentic Loop

Pseudocode Sketch

```
messages = [system_prompt, CLAUDE_MD, task]
tools = [Bash, Read, Edit, Write, Grep,
         Glob, WebFetch, WebSearch, Task]
while not done:
    reply = model.generate(messages,
                           tools=tools)
    if reply.tool_calls:
        for call in reply.tool_calls:
            out = run_tool(call.name,
                           call.args)
            messages.append(
                tool_result(call, out))
    else:
        messages.append(reply.text)
        display(reply.text)
        done = await_user()
```

Examples of Tools

- ▶ **Bash** – run a shell command (pytest -xvs, uv run ..., git log, make)
- ▶ **Read / Write / Edit** – create or open a file, or do a find-and-replace
- ▶ **Grep** – regex-search (e.g. def test_energy across **/*.py)
- ▶ **Glob** – enumerate files by pattern (src/**/*.py)
- ▶ **WebFetch** – pull a specific URL (docs, GitHub issues, papers)
- ▶ **WebSearch** – look up recent/obscure info (post-training data, niche APIs, error message searches)
- ▶ **Task** – spawn a sub-agent with its own context (“audit this PR”, “find all callers of X”). Can use a cheaper model for sub-tasks.

The agent is only as capable as its tools. Add an MCP server and it gains a new action; remove Bash and it can no longer run code. You can teach it to use new CLI tools (or even write new ones which are agent-friendly, with explanatory docs).

Agentic vs. Vibe Coding – Key Differences

- ▶ Agent = model + harness that **executes** code and feeds results back
 - ▶ Core loop is 200 lines of code, not magic (MIT Missing Semester, 2026)
- ▶ **Willison: “Writing code is cheap now”.**
 - ▶ Typing code into a computer is now almost free.
 - ▶ Delivering **good** code, which is tested, correct, maintainable, is not.

For scientific computing, we need to add another dimension:

- ▶ Code must also be physically correct.
- ▶ No auto-generated test suite fully captures that: that is our job.

Common Objections

“AI writes code that doesn’t compile”

Zero-shot, no context, wrong language version: yes, often. With a `CLAUDE.md` specifying your Python/Julia version, package environment, and a working example to follow: almost never. The agent executes and iterates – it sees the compiler error.

“Fortran coverage is low” Correct. Training data is dominated by Fortran 77; modern Fortran 2018 features and fixed/free format confusion are failure modes. My QUIP work in this talk was C and Python glue, not extensively touching the Fortran. Know your tool’s limits.

“Zero-shot failures on niche APIs” Correct.

Zero-shot with no context for a specific LAMMPS fix style or ASE calculator is unreliable. Solution: provide a working example from your codebase. The agent recombines; it doesn’t (shouldn’t) invent.

“I tried it and got rubbish” Almost certainly towards vibe coding end of spectrum: without `CLAUDE.md`, working examples, or iterative feedback. The same person wouldn’t hand a junior developer a task with no briefing and be surprised at the output. Capable models also help — recent flagship models from Anthropic, OpenAI, and Google all clear the bar; older or distilled checkpoints often don’t.

Harder Objections

“It made me slower, not faster” A rigorous RCT (METR, July 2025) found experienced open-source developers were **19% slower** with AI tools. The finding is real for tasks where the developer already knows exactly what to do. Gains appear on unfamiliar territory, bridging expertise gaps, and maintenance work. Track time honestly.

“It’s just regurgitating training data” Partly fair. For novel algorithms, you are the source of novelty: the agent implements what you specify. That is the spec-first pattern: you design, it codes. The Anthropic C compiler rewrite attracted “code laundering” criticism; for original research, the question is whether your idea/spec is novel.

Skill atrophy: the paradox of supervision

Anthropic internal study (Dec 2025): developers scored **17% lower** on code comprehension tests when using AI to learn a new library. Paradox: effective use of Agentic AI requires supervising Claude, which requires the very skills that atrophy. Practise without AI too!

Data privacy and reproducibility Your code goes to Anthropic’s servers – OK for open source code, otherwise not. For sensitive data, local models (Ollama + open weights) are improving rapidly. Hybrid approaches possible.

Pedagogical Tensions

Struggle as pedagogy

- ▶ Hitting a confusing error and working through it **is** the learning.
- ▶ Friction builds intuition.
- ▶ Researchers who never debug never develop a mental model of failure modes.

Reducing friction

- ▶ Remove obstacles that aren't educational.
- ▶ Better tools let people spend cognitive load on the science, not boilerplate.
- ▶ We don't code in assembly any more!
- ▶ AI assistance is now a professional reality in many domains

The question is not whether to use AI tools – it is which friction is worth keeping.

Which friction is worth keeping?

Worth keeping

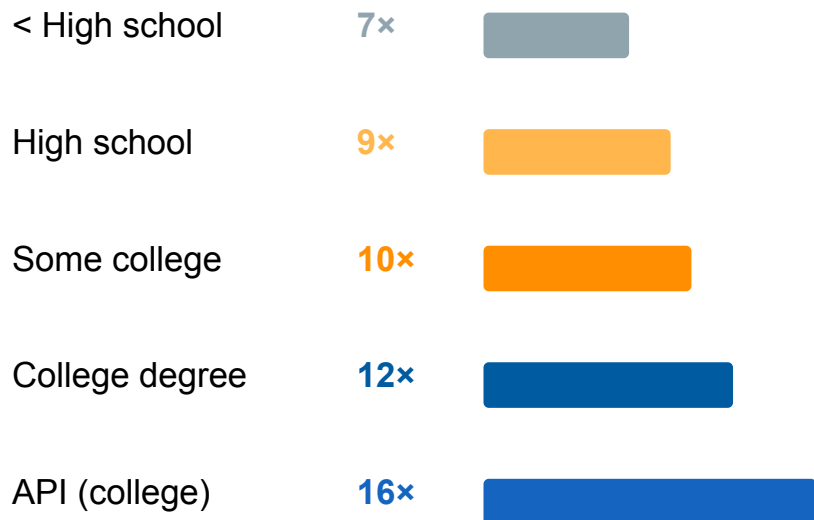
- Debugging logic errors in your own code
- Understanding **why** a numerical method converges or doesn't
- Choosing the right algorithm for the problem
- Interpreting results physically
- Reviewing/owning every line in a paper or PR

Worth removing

- Boilerplate file I/O and plotting scaffolding
- Looking up library API signatures
- Translating a known algorithm into working syntax, or from one language to another
- Writing tests for behaviour you already understand
- Formatting and docstrings

Speedup grows with expertise – Anthropic Economic Index, Jan 2026

Measured speedup on Claude.ai by task complexity *(years of schooling required to understand the task)*



Source: Anthropic Economic Index, Jan 2026 (~2M Claude conversations, Nov 2025) – **vendor-published, treat as directional**.
Speedup = human-alone time / human-with-AI time.

What this means for us as researchers

- ▶ More expertise you bring, more you gain.
- ▶ A PhD student at the frontier gets more leverage than someone doing routine tasks.
- ▶ Contradicts “AI levels the playing field” narrative
- ▶ Domain knowledge is amplified, not devalued
- ▶ The METR 19% slowdown: low end of this curve
- ▶ **Caveat:** data tops out at college-level tasks – no direct measurements of PhD-level work. Trend suggests further amplification; nobody has proved it.

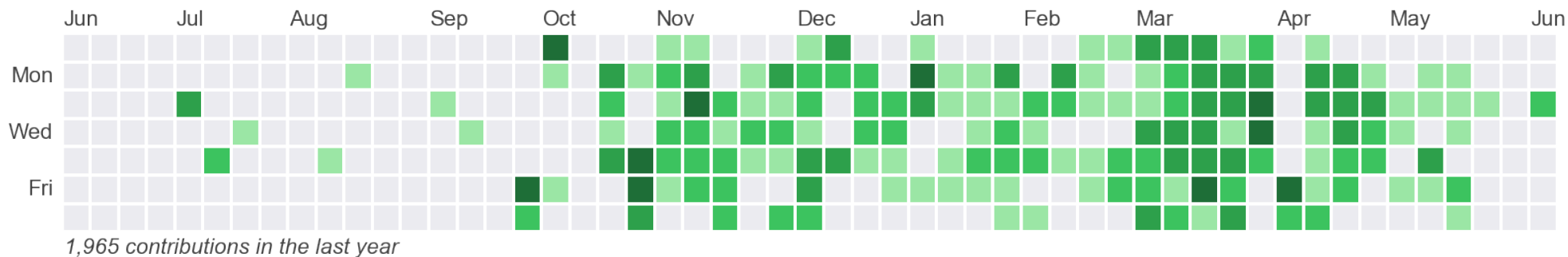
Success rate falls with complexity (70% → 66%).
Human oversight becomes more important, not less.

Research codebases — some worked examples

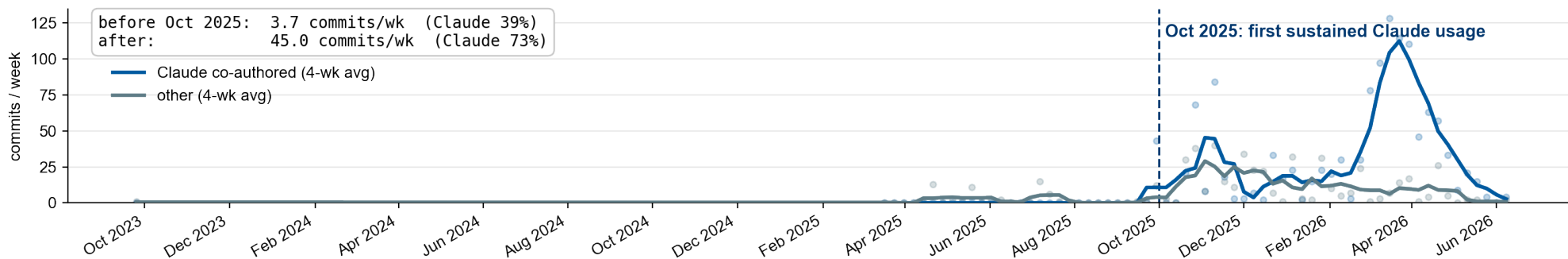
ACEpotentials.jl · LAMMPS interface · GPU port · Atomistica

Can you spot when I started using Claude?

GitHub contribution heatmap (jameskermode · Jun 2025 – Jun 2026)

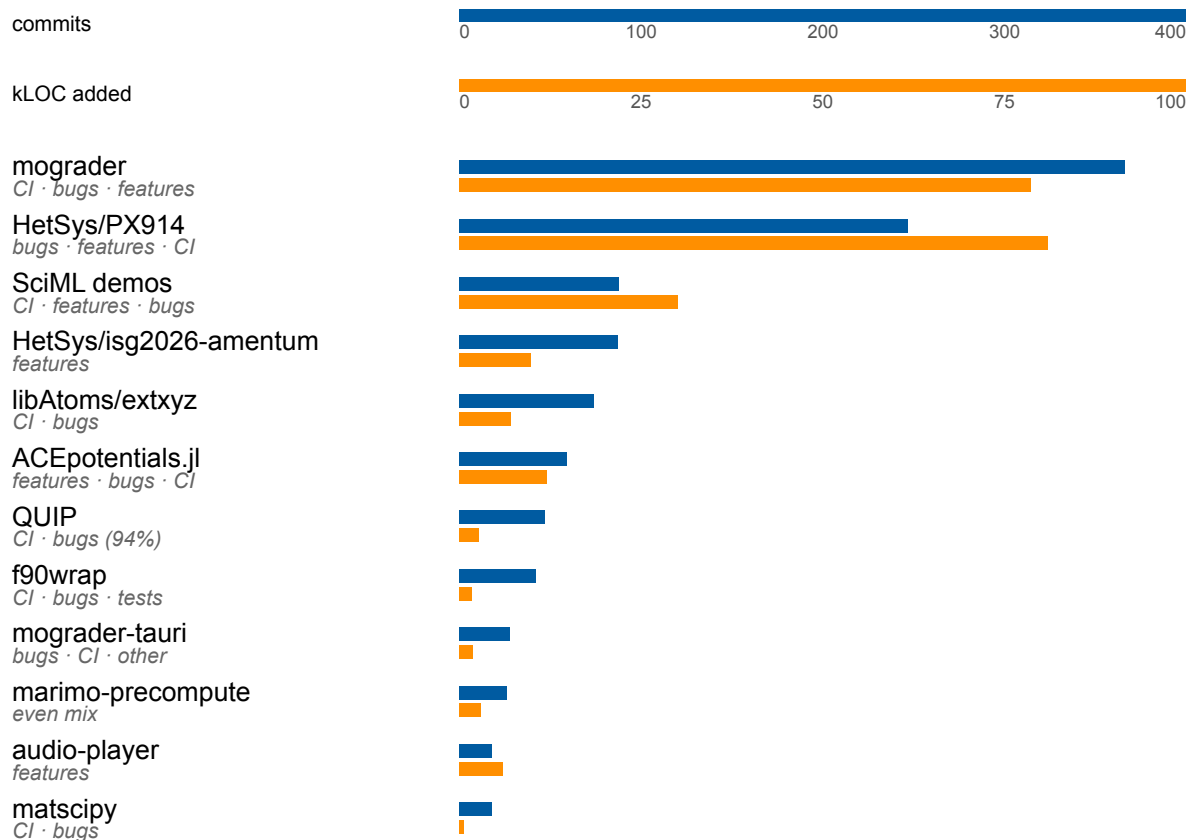


Weekly commits across 36 repos: Claude co-authored vs other



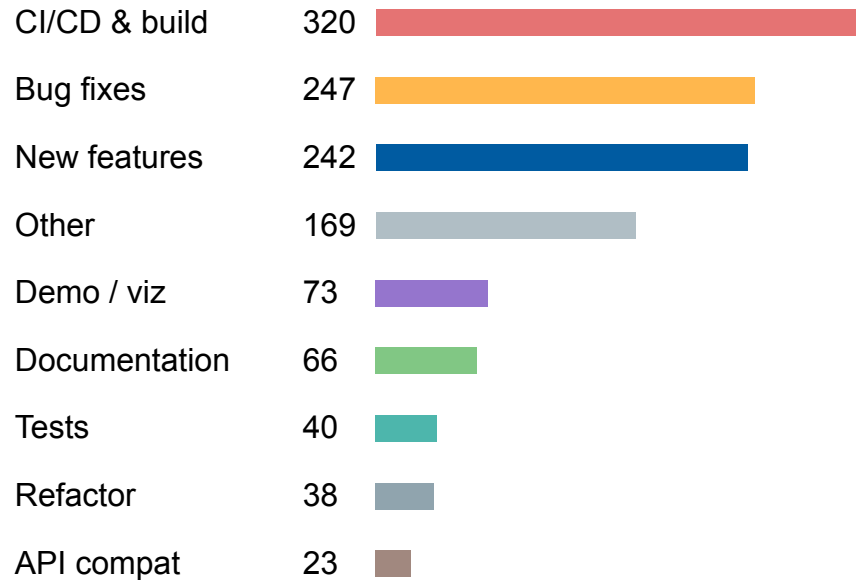
1.2k Claude co-authored commits · 286k lines of code

By repository — commits (blue) and kLOC added (orange)



36 repos · git log --numstat · simulation data excluded · Mar 2025–Jun 2026

By contribution type



26% is CI/CD + build infrastructure – the unglamorous maintenance work that would otherwise never get done.

Cost log: ~£500 over 8 months on Claude Max 5× (~\$2,500 at metered rates). Specialist knowledge made this hard to delegate – counterfactual is “not done”, not “done by someone else”.

ACEpotentials.jl – LAMMPS interface

- ▶ **Goal:** deploy trained ACE potentials in LAMMPS for production MD runs
- ▶ This used to be possible in older ACEpotentials.jl releases, but support was dropped due to lack of maintainer time to keep the ML-PACE C++ code up to date with ACE and LAMMPS changes
- ▶ **Challenge:** bridging Julia and C++, matching LAMMPS pair style conventions

First attempt: `juliac --trim` Native compilation of Julia to a shared library via `juliac --trim --new` niche approach, sparse docs. Agent handled boilerplate and iterated on errors; I knew this was the right strategy and validated the force outputs.

Current: MLIR export Now using MLIR-based compiler pathway with Reactant export to StableHLO. Models callable from Python/JAX or Enzyme C++, giving cleaner interop and better portability. Agent again handled the mechanical translation work; I directed the architectural shift.

Both versions: within **2x of hand-optimised C++** – accomplished in days rather than months. Agent excels at the foreign-function boundary boilerplate that would otherwise take weeks. Good test coverage makes me confident in correctness without needing to analyse every line of the generated code.

GPU port of NeighbourLists.jl

- ▶ Task: GPU-accelerate NeighbourLists.jl for use in ML potential training and inference
- ▶ Baseline: single-threaded CPU implementation – a fair comparison point

Agent contribution

- ▶ Translated Julia neighbour-list to GPU kernels
- ▶ Handled thread indexing and memory layout
- ▶ Generated benchmarking harness
- ▶ Iterated on correctness against CPU reference

Human contribution

- ▶ Understood algorithm well enough to specify it
- ▶ Validated correctness of neighbour lists (periodic BCs, cutoff, odd-shaped cells)
- ▶ Judged when performance was sufficient
- ▶ Integrated into ACEpotentials pipeline

Result: **100x speedup** over single-threaded CPU. Agents surprisingly effective at writing Julia code: mix of high quality training data, language design (e.g. type system).

Three ways Agentic Coding accelerates research

1. Rapid prototyping

The rate-limiting step is often not knowing whether an idea will work. Agentic coding enables testing concepts in hours rather than weeks, failing fast on unpromising directions. This changes the economics of exploration.

2. Bridging expertise gaps

For research that spans disciplines, agentic coding provides working fluency across domains: implementing a sparse GP one day, debugging a LAMMPS Kokkos GPU interface the next.

3. Maintaining complex codebases

Research software accumulates technical debt. Agentic coding handles refactoring, test coverage, CI, and documentation that otherwise gets deferred indefinitely, enabling software releases rather than “code available on request.”

What it won't accelerate

Formulating research questions. Interpreting the physical meaning of results. Knowing when something is subtly wrong. All these require domain expertise.

The wider landscape: AI agents for science

Anthropic: C Compiler (Feb 2026)

16 parallel agents, ~2,000 sessions, \$20k → 100k lines of Rust compiling Linux 6.9. **But** Carlini: “**The thought of programmers deploying software they’ve never personally verified is a real concern.**”

Google: AI Co-Scientist (Feb 2025)

Multi-agent Gemini 2.0 generates novel hypotheses. Drug-repurposing candidates for AML confirmed in vitro; independently reached an AMR hypothesis that took Imperial years. Beyond literature review: novel suggestions.

NVIDIA ALCHEMI (Dec 2025)

NVIDIA Inference Microservices (NIM) for batched conformer search + MD using MACE-MPA-0, TensorNet, AIMNet2. Universal Display: billions of OLED candidates, 10,000x faster than DFT on CPU. Open-source agentic coding toolkit.

Argonne/RSC: agent (2026)

Digital Discovery 2026: autonomous MD pipelines – Atomsk, MLIP web-mining, LAMMPS on HPC, OVITO/Phonopy post-processing. All from natural language. “Run the right simulation” remains human territory.

Agents handle mechanical complexity; domain expertise – what to simulate, whether results make sense – remains the human contribution.

Patterns that work

Four tool-agnostic practices with evidence behind them

Pattern 1 – Hoard things you know how to do

Simon Willison:

“Knowing something is theoretically possible is not the same as having seen it done yourself.”

- ▶ Build a personal library of **working code examples** in your domain
- ▶ For me this means things like: canonical Python examples, code input templates, JAX jit patterns, file readers, neural network inference scripts
- ▶ Reference them in CLAUDE.md or in prompts: “use ~/examples/ace_calc.py as a pattern for this interface”

Agents only need you to figure out a trick once. After that they can recombine it into any similarly shaped future problem, including ones you haven't thought of yet.

Pattern 2 – Spec first, implement second

- ▶ Write the algorithm in pseudocode or maths before opening a session
- ▶ Validate the spec yourself: conserved quantities? edge cases? units?
- ▶ Start a new session to implement – clean context, written spec as the prompt, in planning mode first to force design negotiation before any code is written

```
# SPEC.md – Velocity Verlet integrator
```

```
Input: positions  $r$ , velocities  $v$ , forces  $F$ , timestep  $dt$ 
```

```
1.  $v_{half} = v + 0.5*dt*F/m$ 
```

```
2.  $r_{new} = r + dt*v_{half}$ 
```

```
3. Recompute  $F(r_{new})$ 
```

```
4.  $v_{new} = v_{half} + 0.5*dt*F_{new}/m$ 
```

```
Test: NVE total energy drift <  $1e-4$  eV over 1000 steps (LJ Ar, 94K)
```

Never ask an agent to design the algorithm **and** implement it in one pass.

Pattern 3 – Red/green TDD

- ▶ Write tests that **fail for the right reasons** before any implementation
- ▶ The agent's job is then unambiguous: make the tests pass

For scientific computing, this **forces** you to encode physical correctness explicitly:

```
def test_energy_conservation():  
    traj = run_md(n_steps=1000, ensemble='NVE')  
    drift = max(traj.energies) - min(traj.energies)  
    assert drift < 1e-4 # eV, LJ argon @ 94K
```

- ▶ **You** must specify the tolerance – the agent cannot guess it
- ▶ The test becomes the documentation of what “correct” means

Willison: “TDD produces more succinct and reliable agent output with minimal extra prompting”

Pattern 4 – Manage context obsessively

Context degradation is the primary failure mode. Broad consensus is that output quality starts dropping at ~40% of the context window.

- ▶ `CLAUDE.md`: short, universally applicable – loads on **every** session
- ▶ `/clear` between tasks; `/compact` at natural checkpoints (commits, test passes)
- ▶ Similar tools available for other Agentic Coding systems
- ▶ Maintain `PROGRESS.md` – update at session end, paste in at session start (or use `git log`)

```
# PROGRESS.md
Last session: Zygote AD path implemented, all tests pass
Next: benchmark vs finite-differences on W (110) surface
Known issues: periodic boundary edge case in test_forces_pbc
```

Demo – Claude Code live

Example but somewhat realistic test case

Claude Code demo – pair potential from scratch

The prompt:

Plan then implement a Lennard-Jones pair potential in Python. It should compute energy and forces for an array of pairwise distances. Write a test that checks the force is the negative gradient of the energy using finite differences, to a tolerance of 1e-6. Once validated, interface to ASE Atoms and the matscipy neighbour list, which allows for efficient vectorised operations rather than a quadratic-scaling double loop. Use uv to setup a new venv and manage deps via pyproject.toml. Use red/green TDD with pytest. Ask any clarifications first.

Why the prompt is shaped this way:

- ▶ **Pattern 2 — spec first.** Force the agent to negotiate the design before writing a line.
- ▶ **Pattern 3 — red/green TDD.** You must specify the tolerance; the agent shouldn't guess it.
- ▶ **Staged build-up.** Get the standalone kernel correct in isolation before plumbing it into a framework.
- ▶ **Pattern 1 — hoard.** Hand the agent the names of approaches you already know work.
- ▶ **Pattern 4 — tooling & context.** Conventions baked into the spec (and CLAUDE.md) so the agent doesn't reinvent them.

Here's one I made earlier...

Where I had to steer

- ▶ Claude's first plan was numpy-only; I asked for second JAX-only one. I asked for both hand-derived + autodiff, co-validated. The dual-path design came from that correction – not the prompt.
- ▶ Benchmarks showed native ASE was competitive with JAX. Flagging that as suspicious was my job. The agent measures what you point it at.
- ▶ **Red/green in practice:** every red test was an import or bad kwarg. The 1e-6 FD test passed on first green. No physics bug ever surfaced via TDD.

Where the agent surprised me

- ▶ Its first clarifying question offered the cutoff-shift options unprompted (“shifted: energy continuous at r_c , forces discontinuous”) – **Pattern 2** (spec first) paying off immediately.
- ▶ After the ASE anomaly, Claude diagnosed the cause itself: “19 of 20 MD steps have different pair counts, so JAX re-jits every call” – and implemented pad-to-max-size unprompted.
- ▶ Skills (brainstorming, test-driven-development, verification-before-completion), task lists, and memory notes ran the session autonomously after the plan changes – no /clear needed.

*claude-opus-4-6 · 7 user turns · 116 tool calls · 31 tests · 2 backends agree to 1e-12. **Session cost:** ~\$15 at metered API rates (~\$2–3 amortised on a Max subscription). Equivalent agent work is feasible on any frontier-model harness — Cursor, Codex, Gemini CLI.*

Energy – forward-pass work is on ~623 k **fresh** tokens (cached reads are KV lookups, ~0). Rough guess ~0.1–1 kWh across the session – a few kettle-boils. Dwarfed by one DFT run. JAX+padding kernel is 7.8× faster than ASE native LJ: one MD campaign repays it many times. Repo: github.com/jameskermode/LJ-demo

Anti-patterns

Where it's especially easy to go wrong

Anti-pattern 1 – Unreviewed code on collaborators

Willison: “If you open a PR with code you haven’t validated yourself, you’re delegating your actual work to other people who could have prompted an agent themselves.”

In a research group context:

- ▶ Don’t push simulation code to a shared repo without running it and understanding it
- ▶ Review the **PR description** too: agents write convincing summaries
- ▶ Include evidence of testing: a benchmark plot, diff against reference data, timing numbers

The principle: You are responsible for code with your name on it, regardless of how it was produced: in a paper, a PR, or your thesis.

Anti-pattern 2 – Trusting output you can't verify

The software version: agent writes plausible-looking but broken async logic.

The scientific version is more dangerous:

- ▶ LAMMPS `pair_style` with wrong parameterisation – runs without error, wrong physics
- ▶ ASE constraints that silently don't constrain what you think
- ▶ Units error in force constants: $\text{eV}/\text{\AA}$ vs $\text{eV}/\text{\AA}^2$, off by a factor of $3.8 \text{ \AA}$

Mineault (2026): “A clear skill from scientific training is the ability to call bullshit. Have the agent generate lots of diagnostic plots – individually low-utility, collectively they convince you the data is correct.”

The agent has no notion of physical plausibility. That's up to us.

Anti-pattern 3 – Stateful notebooks

Mineault (2026): “Jupyter notebooks don’t play well with agents: plots are embedded as base64, eating context, and agents don’t know the state of your kernel.”

Jupyter problems

- ▶ Hidden kernel state
- ▶ Base64 plots bloat context window
- ▶ Cell execution order ambiguity
- ▶ Agent can’t verify its own output

Solutions

- ▶ **Marimo**: reactive execution, no hidden state
- ▶ `/marimo-pair` agent tool to edit and run cells
- ▶ CLI scripts + PNGs for data pipelines
- ▶ Notebooks only for final, linear narrative

Anti-pattern 4 – Over-engineering by default

Models are trained to be agreeable. The agent will never tell you a feature is unnecessary, never say “you’re done”, never suggest you stop. Combined with build cost being near-zero, this creates a strong pull toward over-engineering.

Symptoms

- ▶ Speculative features no one asked for
- ▶ Defensive code / fallbacks for cases that can’t happen
- ▶ Premature abstractions and helper layers
- ▶ “While we’re here...” scope creep
- ▶ Endless polish – the agent always finds one more thing

Antidotes

- ▶ Spec the **stop condition** up front; declare done when the spec is met
- ▶ Treat agreeable suggestions skeptically – push back, ask “do we need this?”
- ▶ Periodically prune: delete unused code, undo speculative abstractions
- ▶ Resist the temptation of “just one more iteration”

Outlook – funding & training implications

- ▶ **The substitution ratio.** I've done ~8 months of infrastructure work for £500 of Claude Max subscription (≈ \$2500 at per-token API rates) vs £30–60k of postdoc-equivalent labour. Close to two orders of magnitude.
- ▶ **“Substitution” isn't quite the right frame.** Most of this work required specialist domain knowledge that couldn't be cleanly delegated. Most of it would simply not have been done – closer to “new capability” than “redirected costs”.
- ▶ **Plausible new grant line item:** “AI tooling” alongside HPC compute costs, with consequences for how RSE/PDRA costs are justified and how their day-to-day roles develop.
- ▶ **Where do early-careers learn?** Unglamorous CI / build / maintenance and small new features was always how junior researchers built tacit expertise. If agents do it, the training pipeline needs reworking – not least so the next generation can supervise the agents at all.
- ▶ **Domain expertise is the bottleneck.** Leverage shifts toward people who already know the field, and know what should be possible – with knock-on effects for PhD supervision, PDRA career structures, and the value of institutional knowledge.
- ▶ **Macro version of this:** Anthropic's own “sustained acceleration” scenario imagines 100-person teams doing 10,000-person work (Favaro & Clark, May 2026) – the substitution ratio at lab scale.
- ▶ **Frontier model availability:** Claude Fable withdrawal raises AI sovereignty questions. Can home-grown or open source models catch up/compete?
- ▶ **None of these have settled answers.** We are running an uncontrolled experiment in real time!

Back to Struggle-as-Pedagogy

“Using the tools proficiently is feasible when you have gone through the hard work of writing your own code by yourself, failing repeatedly, picking yourself back up; I don’t know of another way of getting to that level of metacognition.

How do you develop that when the AI is making the mistakes for you, invisibly?

This is likely to be a challenge for junior researchers as the tools get commoditized.”

— Patrick Mineault, **Claude Code for Scientists** (Jan 2026)

Open question:

How do we help new researchers develop metacognition about code when failure is abstracted away?

No settled answer - here’s my partial idea...

Provisional Advice for PhD students

Year 1–2: build intuition first

- ▶ Do the core foundational exercises in training and first research projects manually
- ▶ Friction here is educational: it builds the mental model of failure modes you will need later to supervise an agent
- ▶ Start hoarding working examples as you go (Pattern 1) – future-you will thank present-you!
- ▶ Fine to use AI for boilerplate (plotting, file I/O, CI, docstrings) from day one (in my opinion, check supervisor's view)

Year 3+: leverage, carefully

- ▶ Spec-first becomes essential – novel work means you are the source of novelty, not the agent
- ▶ Delegate the mechanical: input scaffolding, API wrangling, figure plotting, benchmarking harnesses
- ▶ Keep one regular “no-AI” problem for skill maintenance – recall the Dec-2025 Anthropic comprehension result
- ▶ Red/green TDD against **physics** (or other relevant domain), not just code: the agent cannot guess your tolerances

Guidance on Responsible Use of AI in Research

- ▶ Declare substantive AI use following funder, publisher, and university policy: UKRI permits AI usage in proposal writing with caution, Wellcome requires declaration, most publishers want a line in Methods or Code Availability.
- ▶ Most existing policies were written for prose, not code. If generated code shaped a result, disclose it.
- ▶ You are responsible for every equation, figure, and line of code with your name on it. If an agent contributed, you need to understand and verify that contribution.
- ▶ Reproducibility is even more important in the AI age: pinned dependencies, fixed random seeds, good tests. `git log` with Claude trailers is a partial audit trail, I suggest keeping it.
- ▶ Never paste confidential material into standard LLMs without agreement of co-authors/collaborators, because of the risk of data leakage: unpublished manuscripts, industry-partner data, proposals or papers you are reviewing. UKRI specifically prohibits AI in proposal review; most publishers prohibit it for manuscript review.
- ▶ Agree conventions with your supervisor/collaborators early.
- ▶ Check your institution's and funders' AI policies – most now have one. Keep confidential or unpublished data out of assistants that lack an appropriate data-protection agreement.

Summary

Three acceleration mechanisms

- ▶ **Rapid prototyping** – explore and fail fast
- ▶ **Bridging expertise gaps** – fluency across domains
- ▶ **Maintaining codebases** – control technical debt

Four patterns

- ▶ Hoard
- ▶ Spec first
- ▶ Red/green TDD
- ▶ Manage context

Four anti-patterns

- ▶ Dumping unreviewed code on collaborators
- ▶ Trusting output you haven't physically verified
- ▶ Stateful notebooks
- ▶ Over-engineering by default

What Agentic Coding won't accelerate

- ▶ Formulating questions
- ▶ Interpreting physical meaning
- ▶ Knowing when something is subtly wrong

Writing code is cheap now. Delivering **correct** code – and **knowing** it's correct – is not.
Domain expertise is what turns a capable agent into a useful scientific instrument.

In your own work — a QM/MD simulation, an ML model, an analysis of experimental data — where would you trust an agent's output without verification, and where would you never?

What is the difference between those two cases?

Guides & writing

- ▶ **Simon Willison** – Agentic Engineering Patterns (Feb 2026+) simonwillison.net/guides/agentic-engineering-patterns/
- ▶ **Patrick Mineault** – Claude Code for Scientists (Jan 2026) neuroai.science/p/claude-code-for-scientists
- ▶ **MIT Missing Semester** – Agentic Coding (2026) missing.csail.mit.edu/2026/agentic-coding/
- ▶ **Favaro & Clark** – When AI builds itself (May 2026) · *vendor view, directional* anthropic.com/institute/recursive-self-improvement
- ▶ **mograder** – autograder case study, written with agentic coding jameskermode.github.io/mograder/



Slides + source

github.com/jameskermode/agentic-coding-seminar

Released under CC-BY-4.0 · slides themselves produced with Claude Code (see README).

Claude Code plugins

- ▶ **obra/superpowers** – skills framework & methodology github.com/obra/superpowers
- ▶ **marimo-team/marimo-pair** – agents inside a marimo notebook github.com/marimo-team/marimo-pair